

INSTITUTO DE PÓS-GRADUAÇÃO EM ENGENHARIA DE SOFTWARE
CURSO DE PÓS-GRADUAÇÃO LATO SENSU — MBA EM ENGENHARIA DE
SOFTWARE

ENGENHARIA DE SOFTWARE ASSISTIDA POR
INTELIGÊNCIA ARTIFICIAL:
do desenvolvedor codificador ao orquestrador de agentes — prompt,
contexto e harness como novas disciplinas de engenharia

Autor: [Nome do(a) Mestrando(a)]

Orientador(a): [Nome do(a) Orientador(a)]

Cidade — Brasil

2026

[NOME DO(A) MESTRANDO(A)]

**ENGENHARIA DE SOFTWARE ASSISTIDA POR INTELIGÊNCIA
ARTIFICIAL:**

do desenvolvedor codificador ao orquestrador de agentes

Monografia apresentada ao Curso de Pós-Graduação Lato Sensu (MBA) em Engenharia de Software como requisito parcial para obtenção do título de Especialista, sob orientação do(a) Prof.(a) [Nome], submetida à defesa perante banca examinadora.

Cidade — Brasil

2026

RESUMO

A consolidação de modelos de linguagem de grande porte (Large Language Models — LLMs) como ferramentas cotidianas de desenvolvimento provocou um deslocamento estrutural na prática da engenharia de software: a complexidade migra da escrita de código para a especificação, a governança e a orquestração de sistemas que escrevem código. Esta monografia tem por objetivo sistematizar, em chave de engenharia, o conjunto de conceitos, técnicas e arquiteturas que sustentam o desenvolvimento de software assistido por IA, organizando-os em três disciplinas emergentes e complementares — engenharia de prompt (prompt engineering), engenharia de contexto (context engineering) e engenharia de ambiente operacional (harness engineering) — e relacionando-os com fundamentos consolidados da área, que permanecem necessários. Adota-se metodologia de pesquisa qualitativa, de natureza teórico-conceitual, apoiada em revisão de literatura técnico-científica e na análise de práticas correntes da indústria. Discutem-se: a redefinição do papel do desenvolvedor, de codificador para coautor e orquestrador; as técnicas de prompting (zero-shot, few-shot, cadeia de pensamento e ReAct); a gestão da janela de contexto e seus limites de degradação; o Model Context Protocol (MCP) e o conceito de skills como mecanismos de extensão e modularização do contexto; o desenvolvimento orientado a especificações (Spec-Driven Development); a paralelização do trabalho por meio de árvores de trabalho do Git e de agentes de revisão; e as arquiteturas de agente único e multiagente, incluindo a geração aumentada por recuperação (Retrieval-Augmented Generation — RAG). Conclui-se que a engenharia de software assistida por IA não dispensa, mas intensifica, a exigência de fundamentos sólidos, e que as três disciplinas propostas constituem um arcabouço útil para a formação profissional e para a condução disciplinada de projetos na nova realidade produtiva.

Palavras-chave: Engenharia de software. Inteligência artificial. Modelos de linguagem. Engenharia de contexto. Agentes de IA.

ABSTRACT

The consolidation of Large Language Models (LLMs) as everyday software development tools has triggered a structural shift in software engineering practice: complexity migrates from writing code to specifying, governing, and orchestrating systems that write code. This monograph aims to systematize, from an engineering standpoint, the set of concepts, techniques, and architectures underlying AI-assisted software development, organizing them into three emerging and complementary disciplines — prompt engineering, context engineering, and harness engineering — and relating them to consolidated foundations of the field, which remain necessary. A qualitative, theoretical-conceptual research methodology is adopted, supported by a review of technical-scientific literature and by the analysis of current industry practices. The work discusses: the redefinition of the developer role, from coder to co-author and orchestrator; prompting techniques (zero-shot, few-shot, chain-of-thought, and ReAct); context window management and its degradation limits; the Model Context Protocol (MCP) and the notion of skills as mechanisms for context extension and modularization; Spec-Driven Development; work parallelization through Git worktrees and review agents; and single-agent and multi-agent architectures, including Retrieval-Augmented Generation (RAG). It concludes that AI-assisted software engineering does not waive but rather intensifies the requirement for solid fundamentals, and that the three proposed disciplines constitute a useful framework for professional education and for the disciplined conduct of projects in the new productive reality.

Keywords: Software engineering. Artificial intelligence. Language models. Context engineering. AI agents.

*A todos os profissionais que, diante de cada
mudança de paradigma, escolhem aprender em vez
de temer.*

AGRADECIMENTOS

Ao(À) orientador(a), pela condução criteriosa e pelas observações que delimitaram o escopo deste trabalho. Aos professores da banca examinadora, pela disponibilidade em avaliar e aprimorar a pesquisa. Aos colegas de profissão, cujas discussões sobre a prática cotidiana do desenvolvimento assistido por inteligência artificial forneceram o substrato empírico que esta monografia procurou sistematizar. E à família, pelo apoio ao longo de todo o percurso acadêmico.

“Novos poderes trazem novas responsabilidades.”

*Adágio da engenharia, aplicável a cada nova ferramenta que amplia
o alcance — e os riscos — do ofício.*

LISTA DE ABREVIATURAS E SIGLAS

- ADK** — Agent Development Kit
- ADR** — Architecture Decision Record (Registro de Decisão de Arquitetura)
- API** — Application Programming Interface (Interface de Programação de Aplicações)
- DDD** — Domain-Driven Design (Projeto Orientado ao Domínio)
- GoF** — Gang of Four (autores de Design Patterns, 1994)
- IA** — Inteligência Artificial
- LLM** — Large Language Model (Modelo de Linguagem de Grande Porte)
- MCP** — Model Context Protocol (Protocolo de Contexto de Modelo)
- PRD** — Product Requirements Document (Documento de Requisitos de Produto)
- RAG** — Retrieval-Augmented Generation (Geração Aumentada por Recuperação)
- ReAct** — Reasoning and Acting (Raciocínio e Ação)
- RPI** — Research, Plan, Implement (Pesquisar, Planejar, Implementar)
- SDD** — Spec-Driven Development (Desenvolvimento Orientado a Especificações)
- SRE** — Site Reliability Engineering (Engenharia de Confiabilidade de Sistemas)
- TDD** — Test-Driven Development (Desenvolvimento Guiado por Testes)

SUMÁRIO

1	Introdução.....	13
1.1	Contextualização.....	13
1.2	Problema de pesquisa.....	14
1.3	Objetivos.....	14
1.3.1	Objetivo geral.....	14
1.3.2	Objetivos específicos.....	14
1.4	Justificativa.....	15
1.5	Metodologia.....	15
1.6	Estrutura do trabalho.....	15
2	O Novo Paradigma do Desenvolvimento de Software.....	16
2.1	As três camadas da inteligência artificial aplicada.....	16
2.2	A inteligência artificial como ferramenta de engenharia.....	16
2.3	Do codificador ao coautor e orquestrador.....	17
2.4	O deslocamento da complexidade para a especificação.....	17
2.5	Permanência dos fundamentos e surgimento de novos paradigmas.....	18
3	Engenharia de Prompt.....	19
3.1	Definição e alcance.....	19
3.2	Prompting zero-shot.....	19
3.3	Prompting few-shot.....	20
3.4	Cadeia de pensamento (Chain-of-Thought).....	20
3.5	ReAct: raciocínio e ação.....	20
3.6	Critério de seleção e o parâmetro de temperatura.....	21
4	Engenharia de Contexto.....	22
4.1	Da engenharia de prompt à engenharia de contexto.....	22
4.2	Tokens, embeddings e a janela de contexto.....	22
4.3	Degradação de desempenho e alucinação.....	22
4.4	Persistência de estado entre sessões.....	23
4.5	Model Context Protocol (MCP).....	23
4.6	Skills: conhecimento estendido e modular.....	24
4.7	Estudo ilustrativo: a evolução do desenvolvimento de interfaces.....	24
4.8	Validação assistida por ferramentas.....	25
5	Engenharia de Ambiente Operacional (Harness).....	26

5.1 As três camadas: prompt, contexto e harness.....	26
5.2 Harness embutido e harness do projeto.....	26
5.3 Arquivos de diretrizes e regras.....	27
5.4 Laços de realimentação (feedback loops).....	27
5.5 Segurança no ambiente operacional.....	28
5.6 Síntese: o harness como camada de governança.....	28
6 Desenvolvimento Orientado a Especificações.....	28
6.1 Motivação: os limites do planejamento livre.....	28
6.2 A esteira de refinamento.....	29
6.3 Ferramentas de SDD e o conceito de constituição.....	29
6.4 A malha de documentos gerada.....	30
6.5 Rastreamento de progresso e gestão de contexto.....	30
7 Paralelização do Trabalho e Automação do Ciclo de Entrega.....	31
7.1 Do desenvolvimento sequencial ao paralelo.....	31
7.2 Árvores de trabalho do Git (git worktrees).....	31
7.3 Planejamento de dependências entre tarefas.....	32
7.4 Agentes de revisão e automação da integração.....	32
8 Arquiteturas de Agentes de Inteligência Artificial.....	33
8.1 O que é, e o que não é, um agente.....	33
8.2 Arquitetura de agente único.....	33
8.3 Arquitetura multiagente.....	34
8.4 Trade-offs entre as arquiteturas.....	34
8.5 Economia de tokens em aplicações agênticas.....	35
8.6 Frameworks de desenvolvimento de agentes.....	35
8.7 O padrão agente-como-ferramenta.....	36
8.8 Geração aumentada por recuperação (RAG).....	36
8.9 Ilustração: aplicação multiagente com RAG.....	37
8.10 Memória e a colisão entre mecanismos.....	37
9 Segurança, Qualidade e Governança.....	38
9.1 Novos vetores de ameaça.....	38
9.2 Guardrails.....	38
9.3 Qualidade, avaliação e custo.....	39
9.4 Observabilidade e governança da automação.....	39
10 Implicações para a Formação e o Mercado de Trabalho.....	39

10.1 Fundamentos e o desafio do profissional iniciante.....	39
10.2 Reciclagem contínua e ruído informacional.....	40
10.3 Adoção proporcional e contexto econômico.....	40
11 Considerações Finais.....	41
Referências.....	42

1 Introdução

1.1 Contextualização

A engenharia de software é uma disciplina historicamente marcada por mudanças de paradigma. Da programação estruturada à orientação a objetos, da web monolítica aos microsserviços, cada onda tecnológica redefiniu não apenas as ferramentas, mas o próprio conjunto de competências exigidas do profissional. A emergência e a rápida popularização dos modelos de linguagem de grande porte (Large Language Models — LLMs), apoiados na arquitetura Transformer (VASWANI et al., 2017) e amadurecidos pela demonstração de capacidades de aprendizado a partir de poucos exemplos (BROWN et al., 2020), inauguram uma transformação de magnitude comparável — porém com uma característica distintiva: pela primeira vez, a ferramenta de produção é capaz de gerar o próprio artefato central do ofício, o código-fonte.

A consequência imediata é a desmaterialização do esforço de codificação. Atividades que durante décadas constituíram o núcleo do trabalho do desenvolvedor — traduzir requisitos em sintaxe, memorizar interfaces de programação de aplicações (APIs), escrever laços e condicionais, depurar manualmente — passam a ser delegáveis, com grau crescente de autonomia, a sistemas de IA. Esse movimento não torna o desenvolvedor obsoleto; ao contrário, desloca o eixo de valor de sua atuação. A escrita de código aproxima-se da condição de commodity, enquanto a capacidade de especificar com precisão, de arquitetar soluções, de governar a qualidade e de orquestrar sistemas autônomos torna-se o diferencial profissional.

Essa transição, contudo, é frequentemente vivida pelos profissionais de modo desordenado. A profusão de termos — prompt engineering, context engineering, harness, agentes, MCP, skills, Spec-Driven Development, RAG — sucede-se em ondas de entusiasmo que dificultam a distinção entre conceitos estruturantes e modismos passageiros. Há, portanto, demanda legítima por uma sistematização que organize esse vocabulário em chave de engenharia, isto é, que o conecte a fundamentos, a trade-offs explícitos e a práticas verificáveis.

1.2 Problema de pesquisa

O problema que orienta este trabalho pode ser enunciado na seguinte questão: como organizar, de forma coerente e fundamentada, o conjunto de conceitos e técnicas do desenvolvimento de software assistido por IA, de modo a sustentar tanto a formação profissional quanto a condução disciplinada de projetos? Subjazem a essa questão três tensões recorrentes. A primeira é a tensão entre automação e controle: quanto delegar à IA sem perder a capacidade de auditar e responder pelo resultado. A segunda é a tensão entre novidade e fundamento: em que medida os princípios consolidados da engenharia de software permanecem válidos. A terceira é a tensão entre produtividade e custo, tanto financeiro quanto cognitivo, da introdução de IA no fluxo de trabalho.

1.3 Objetivos

1.3.1 Objetivo geral

Sistematizar, em perspectiva de engenharia, os conceitos, técnicas e arquiteturas que fundamentam o desenvolvimento de software assistido por inteligência artificial, propondo sua organização em três disciplinas complementares — engenharia de prompt, engenharia de contexto e engenharia de ambiente operacional (harness).

1.3.2 Objetivos específicos

- Caracterizar a redefinição do papel do desenvolvedor e a recolocação da complexidade do código para a especificação e a orquestração.
- Descrever as principais técnicas de engenharia de prompt e seus respectivos modos de raciocínio, custos e indicações de uso.
- Analisar a janela de contexto como recurso finito, seus limites de degradação e os mecanismos de extensão (Model Context Protocol) e modularização (skills).
- Conceituar a engenharia de ambiente operacional (harness) e seus laços de realimentação como camada de governança da automação.
- Apresentar o desenvolvimento orientado a especificações (Spec-Driven Development) e a paralelização do trabalho com agentes e árvores de trabalho do Git.
- Examinar as arquiteturas de agente único e multiagente, seus trade-offs e os frameworks correntes, incluindo a geração aumentada por recuperação (RAG).

- Discutir implicações de segurança, qualidade, custo e formação decorrentes da adoção dessas práticas.

1.4 Justificativa

A relevância do tema é simultaneamente acadêmica e prática. No plano acadêmico, há um descompasso entre a velocidade de adoção das ferramentas de IA na indústria e a consolidação de um corpo conceitual que as organize segundo critérios de engenharia. Termos cunhados em blogs, repositórios e conferências circulam sem definição precisa, o que dificulta o ensino e a comunicação rigorosa. No plano prático, a ausência de sistematização conduz a dois extremos igualmente improdutivos: a rejeição das ferramentas, que sacrifica ganhos expressivos de produtividade, e a adoção acrítica, que gera código frágil, custos descontrolados e vulnerabilidades de segurança. Uma abordagem de engenharia — que reconheça trade-offs, exija verificação e preserve fundamentos — é a via capaz de mediar esses extremos.

1.5 Metodologia

Adota-se metodologia de pesquisa qualitativa, de natureza teórico-conceitual e finalidade aplicada. O trabalho não constitui estudo experimental controlado; antes, organiza e interpreta um campo em formação. Os procedimentos compreendem: (i) revisão de literatura técnico-científica, privilegiando fontes primárias verificáveis — artigos seminais sobre modelos de linguagem e técnicas de prompting, obras consolidadas de engenharia de software e especificações de protocolos abertos; (ii) análise conceitual, voltada à definição precisa de termos e ao mapeamento de suas relações; e (iii) sistematização das práticas correntes da indústria, descritas em nível de padrão e de trade-off, sem vinculação a fornecedores específicos. Quando se mencionam relatos de incidentes ou de adoção corporativa, eles são apresentados de forma generalizada, como ilustração de padrões documentados, e não como evidência empírica primária deste estudo.

1.6 Estrutura do trabalho

Além desta introdução, a monografia organiza-se em dez capítulos. O Capítulo 2 examina o novo paradigma do desenvolvimento e a redefinição do papel do desenvolvedor. O Capítulo 3 trata da engenharia de prompt e de suas técnicas. O Capítulo 4 aborda a engenharia

de contexto, a janela de contexto, o MCP e as skills. O Capítulo 5 conceitua a engenharia de ambiente operacional (harness). O Capítulo 6 apresenta o desenvolvimento orientado a especificações. O Capítulo 7 discute a paralelização do trabalho e a automação do ciclo de entrega. O Capítulo 8 dedica-se às arquiteturas de agentes de IA e ao RAG. O Capítulo 9 trata de segurança, qualidade e governança. O Capítulo 10 discute as implicações para a formação e o mercado. O Capítulo 11 apresenta as considerações finais.

2 O Novo Paradigma do Desenvolvimento de Software

2.1 As três camadas da inteligência artificial aplicada

Para situar com precisão o lugar do desenvolvedor de aplicações na cadeia de valor da IA, é útil distinguir três camadas. A primeira é a camada de infraestrutura, que compreende o hardware especializado (unidades de processamento gráfico e tensorial), os provedores de computação em nuvem e o provisionamento que sustenta o treinamento e a inferência. A segunda é a camada de treinamento de modelos, domínio de engenheiros de aprendizado de máquina e cientistas de dados, na qual se decidem arquiteturas, conjuntos de dados, estratégias de ajuste fino (fine-tuning) e avaliação. A terceira é a camada de desenvolvimento de aplicações, em que o profissional consome os modelos como serviço para construir software que resolve problemas de negócio.

A delimitação é importante porque define competências e responsabilidades. O desenvolvedor de aplicações é, antes de tudo, usuário dos modelos produzidos na camada intermediária, os quais, por sua vez, dependem da infraestrutura da camada de base. Não lhe compete dominar algoritmos de treinamento ou decisões de provisionamento de hardware, mas sim as disciplinas próprias de sua camada: a engenharia de prompt, a engenharia de contexto, os protocolos de comunicação entre modelos e ferramentas, e a construção e orquestração de agentes. É nessa camada que esta monografia se concentra.

2.2 A inteligência artificial como ferramenta de engenharia

Reconhecida a camada de atuação, a IA passa a ser compreendida não como substituta do engenheiro, mas como ferramenta que potencializa atividades preexistentes ao longo de todo o ciclo de vida do software. Entre os usos consolidados, destacam-se: a análise de problemas e a pesquisa de alternativas técnicas com seus trade-offs; o apoio à tomada de

decisões de arquitetura; a geração de código; a identificação de defeitos e a revisão automatizada; a documentação; e a construção de pipelines capazes de executar tarefas com graus variados de autonomia. O efeito combinado é duplo: ganho de produtividade em atividades já existentes e transformação qualitativa de algumas delas, como a revisão de código, que deixa de ser exclusivamente manual.

2.3 Do codificador ao coautor e orquestrador

A redefinição mais profunda recai sobre o papel do profissional. O desenvolvedor que se identificava primordialmente pela escrita de código — pela fluência sintática, pela memorização de bibliotecas, pela destreza no terminal — vê esse núcleo de identidade erodir. Em seu lugar emerge o papel de coautor e orquestrador: aquele que especifica, pilota, revisa e valida. A metáfora do maestro é elucidativa: o profissional rege a execução, distribuindo intenções a executores — os agentes de IA — e respondendo pela coerência do todo.

Essa mudança tem corolários relevantes. Diminui a necessidade de digitar código e de memorizar interfaces, mas aumenta, em contrapartida, a exigência de compreensão do domínio do negócio, de capacidade de especificação e de produção de documentação técnica — documentos de requisitos de produto (PRD), registros de decisão de arquitetura (ADR), diagramas de contexto. O perfil profissional isolado, que concentrava conhecimento e evitava a interação, torna-se disfuncional; o novo perfil é necessariamente colaborativo, dialogando com pessoas de produto, gestão e negócio. Surge a designação, já corrente no mercado, de engenheiro de IA (AI engineer), que sintetiza a expansão de horizontes imposta pela ferramenta: barreiras antes rígidas entre front-end, back-end e operações tornam-se mais permeáveis, pois a IA amplia o alcance de cada profissional.

2.4 O deslocamento da complexidade para a especificação

O ponto central do novo paradigma é o deslocamento da complexidade. Quando a geração de código se torna barata e veloz, o gargalo da qualidade migra para montante, para a especificação. Uma especificação ambígua, incompleta ou contraditória, consumida como contexto pela IA, produz código defeituoso; e código defeituoso gerado em escala produz problemas em escala. Daí a máxima que organiza a prática: a implementação é, em boa medida, um detalhe a jusante de um planejamento bem feito. O esforço intelectual que antes

se concentrava na codificação realoca-se para a concepção, a validação de critérios e a definição de contratos.

Cabe uma ressalva de escopo. As práticas aqui sistematizadas dirigem-se sobretudo a aplicações de porte significativo — com múltiplos usuários, restrições explícitas, processos de negócio complexos e exigências de manutenção a longo prazo. Para software de baixa complexidade e vida curta, grande parte do aparato descrito é desnecessária. A distinção evita um erro comum: aplicar a sistemas simples uma sobre-engenharia que só se justifica em sistemas críticos.

2.5 Permanência dos fundamentos e surgimento de novos paradigmas

Se muito mudou, é igualmente verdadeiro que os fundamentos consolidados da engenharia de software não apenas permanecem válidos como se tornam mais necessários. Princípios de projeto orientado a objetos, arquitetura limpa, projeto orientado ao domínio (EVANS, 2003), boas práticas de codificação (MARTIN, 2008) e desenvolvimento guiado por testes (BECK, 2002) constituem precisamente o repertório que habilita o profissional a especificar, revisar e validar o que a IA produz. Sem fundamento sólido, o coautor não tem como guiar o agente nem como auditar o resultado. A responsabilidade por um defeito introduzido permanece humana: não se atribui a falha à ferramenta, mas a quem a operou.

Simultaneamente, novos paradigmas surgem como contrapartes dos clássicos. Aos protocolos de comunicação tradicionais (HTTP, WebSocket) somam-se protocolos voltados à interação entre modelos e ferramentas, como o Model Context Protocol (MCP). À metodologia dos doze fatores para aplicações nativas de nuvem (WIGGINS, 2011) corresponde uma proposta análoga de boas práticas para a construção de agentes (HUMANLAYER, 2024). Aos padrões de projeto clássicos (GAMMA et al., 1994) somam-se padrões agênticos, como roteamento, uso de ferramentas e o ciclo de raciocínio e ação (YAO et al., 2023). E ao estilo de microsserviços (LEWIS; FOWLER, 2014) corresponde a arquitetura multiagente, na qual unidades autônomas cooperam para resolver um problema. O Quadro 1 resume essas correspondências.

Quadro 1 — Correspondência entre fundamentos clássicos e novos paradigmas.

Fundamento clássico	Contraparte na era da IA
Protocolos HTTP/WebSocket	MCP e protocolos de interação agente–ferramenta

The Twelve-Factor App (2011)	12-Factor Agents
Padrões de projeto (GoF, 1994)	Padrões agênticos (Tools, Routing, ReAct)
Microsserviços	Arquitetura multiagente
SOLID, Clean Code, DDD, TDD	Permanecem — base para especificar e validar

Fonte: elaborado pelo autor.

3 Engenharia de Prompt

3.1 Definição e alcance

Engenharia de prompt (prompt engineering) é a disciplina que estuda a formulação das instruções fornecidas a um modelo de linguagem com vistas a obter respostas adequadas, confiáveis e reproduzíveis. Há um equívoco recorrente em reduzi-la a "saber pedir". Embora a clareza do pedido seja relevante, a disciplina abrange um conjunto de técnicas com estruturas e modos de raciocínio distintos, frequentemente combinadas. No contexto do desenvolvimento de software, a engenharia de prompt sustenta atividades como exploração e contextualização de problemas, pesquisa de alternativas, geração de especificações e de código, definição de fluxos de trabalho, expansão de tarefas em subtarefas, geração de testes para casos-limite, depuração assistida, produção de mensagens de commit no padrão de commits convencionais, abertura de requisições de incorporação (pull requests) e revisão de código.

Surveys recentes catalogam dezenas de técnicas e suas variações (SCHULHOFF et al., 2024), o que evidencia a maturidade crescente do campo. Para os fins desta monografia, concentram-se quatro técnicas de uso frequente no desenvolvimento — zero-shot, few-shot, cadeia de pensamento e ReAct — escolhidas por exemplificarem fluxos de raciocínio qualitativamente diferentes.

3.2 Prompting zero-shot

No prompting zero-shot, formula-se uma pergunta ou tarefa direta, sem exemplos e sem orientação sobre o caminho de resolução. O modelo processa a instrução e devolve a resposta que considera mais provável. É a técnica mais econômica em tokens e a mais rápida, adequada a tarefas simples e bem delimitadas. Seu risco é a baixa controlabilidade: na

ausência de acesso a dados externos, o modelo pode preencher lacunas com informação plausível porém inexistente — fenômeno conhecido como alucinação. Em uma tarefa de triagem que exija, por exemplo, identificadores de registros reais, o zero-shot tenderá a inventá-los.

3.3 Prompting few-shot

O prompting few-shot acrescenta ao pedido um pequeno número de exemplos resolvidos, que orientam o modelo quanto ao formato e ao critério esperados. A demonstração de que modelos de linguagem suficientemente grandes aprendem tarefas a partir de poucos exemplos, sem ajuste de pesos, é um dos resultados fundadores do campo (BROWN et al., 2020). Em engenharia de software, o few-shot é útil quando se deseja padronizar a saída — um estilo de mensagem de commit, um formato de resposta, uma convenção de nomenclatura — fornecendo um ou dois casos modelo.

3.4 Cadeia de pensamento (Chain-of-Thought)

A técnica de cadeia de pensamento (chain-of-thought) instrui o modelo a explicitar o raciocínio em etapas antes de apresentar a resposta final. Em vez de saltar ao resultado, o modelo decompõe o problema, encadeia inferências parciais, verifica e conclui. Wei et al. (2022) demonstraram que essa explicitação melhora substancialmente o desempenho em tarefas que exigem raciocínio aritmético, simbólico e de senso comum. Para o desenvolvimento, a técnica é valiosa na análise de funcionalidades complexas e no levantamento de casos-limite, conferindo maior profundidade e assertividade. Uma vantagem prática é o custo: o raciocínio em etapas ocorre, tipicamente, em uma única chamada ao modelo, o que a torna econômica em relação a abordagens iterativas.

3.5 ReAct: raciocínio e ação

A técnica ReAct (Reasoning and Acting) intercala raciocínio e ação em um ciclo iterativo de pensamento, ação e observação (YAO et al., 2023). Sua característica distintiva é que, na etapa de ação, o modelo pode invocar ferramentas externas — consultar um banco de dados, pesquisar na web, ler um diretório — e observar o resultado real antes de prosseguir. O ciclo repete-se até que o modelo julgue ter informação suficiente para responder, momento em que encerra o laço. Diferentemente das técnicas anteriores, o ReAct tende a exigir

múltiplas chamadas, sendo mais lento e mais custoso; em contrapartida, introduz uma etapa de verificação contra dados reais que reduz a alucinação. As interfaces de linha de comando dos agentes de codificação contemporâneos apoiam-se largamente nesse padrão.

Convém observar a distinção conceitual entre o prompt e a capacidade de agir. O prompt, isoladamente, não executa nada: ele apenas estrutura a instrução. A capacidade de invocar ferramentas e injetar seus resultados no contexto provém da infraestrutura que envolve o modelo — tema da engenharia de ambiente operacional, discutida no Capítulo 5. O ReAct, portanto, articula a engenharia de prompt com essa infraestrutura.

3.6 Critério de seleção e o parâmetro de temperatura

A escolha da técnica obedece à natureza da tarefa. Quando não há dependência de dados externos, empregam-se zero-shot (eventualmente acrescido de exemplos, no few-shot) ou cadeia de pensamento e suas variações. Quando há dependência de ferramentas ou de informação externa, o ReAct é a opção natural. Essas técnicas não são mutuamente exclusivas: um único fluxo pode combinar few-shot para fixar o formato e ReAct para a execução.

Um parâmetro operacional relevante é a temperatura, que regula o grau de aleatoriedade da geração. Valores próximos de zero produzem respostas mais determinísticas e fiéis à instrução, recomendáveis em tarefas de classificação e extração; valores mais altos ampliam a variabilidade, úteis em tarefas criativas. Em pipelines de desenvolvimento que exigem reprodutibilidade, a temperatura baixa é, em geral, preferível.

A título ilustrativo, considere-se uma tarefa hipotética de triagem de chamados de suporte. Sob zero-shot, o modelo classifica o sentimento e a prioridade, mas, ao ser solicitado a listar chamados similares sem acesso à base, inventa identificadores. Sob cadeia de pensamento, a classificação ganha profundidade pela decomposição explícita, mas a limitação quanto a dados externos persiste. Sob ReAct, o modelo consulta a base real por meio de uma ferramenta de busca, recupera chamados efetivamente existentes e produz uma saída estruturada e verificável. O exemplo evidencia como a escolha da técnica determina a confiabilidade do resultado.

4 Engenharia de Contexto

4.1 Da engenharia de prompt à engenharia de contexto

Se a engenharia de prompt cuida do que se pede ao modelo, a engenharia de contexto (context engineering) cuida do que o modelo vê. O contexto não se resume ao prompt do usuário: inclui o prompt de sistema, os arquivos de regras do projeto, trechos de código-fonte, resultados de ferramentas, histórico da conversa e quaisquer dados injetados ao longo da interação. Engenharia de contexto é, portanto, a disciplina de decidir qual conteúdo deve ocupar a janela de contexto do modelo, em que momento e em que volume. Trata-se de um problema de seleção sob restrição, e não de mera acumulação.

4.2 Tokens, embeddings e a janela de contexto

Modelos de linguagem não operam diretamente sobre texto, mas sobre tokens — unidades subpalavra obtidas por um processo de tokenização e convertidas em vetores numéricos (embeddings) que codificam relações semânticas. Cada interação consome tokens de entrada (o conteúdo enviado) e produz tokens de saída (a resposta gerada), e ambos são tarifados. A janela de contexto é a quantidade máxima de tokens que o modelo consegue considerar simultaneamente. Os modelos contemporâneos ampliaram significativamente esse limite, alcançando, em algumas famílias, a ordem de centenas de milhares a um milhão de tokens; tais números, contudo, variam por fornecedor e versão e devem ser consultados na documentação oficial, pois evoluem rapidamente.

O histórico da conversa é reenviado a cada nova interação, de modo que os tokens se acumulam progressivamente. Mecanismos de cache mitigam a latência e o custo desse reenvio, mas o crescimento do contexto permanece um fator central de gestão.

4.3 Degradação de desempenho e alucinação

A existência de uma janela ampla não significa que se deva preenchê-la integralmente. Modelos de linguagem apresentam degradação de desempenho muito antes do limite nominal: à medida que o contexto cresce, a capacidade de localizar e utilizar a informação relevante diminui. Liu et al. (2023) documentaram que o desempenho é sensível à posição da informação no contexto, com queda notável quando dados relevantes se encontram no meio

de contextos longos — fenômeno sintetizado na expressão "perdido no meio". Esse resultado sustenta, em caráter geral, a recomendação de manter o contexto enxuto.

Na prática da indústria, adota-se a heurística — não derivada de um limiar teórico, mas da observação empírica e dependente de cada modelo — de que a qualidade começa a declinar quando o contexto ultrapassa aproximadamente 40% da capacidade, e de que, próximo a 80%, o cenário torna-se crítico, pois o próprio modelo precisa reservar tokens para compactar o histórico. Quando essa faixa é atingida, a compactação tende a ser de baixa qualidade, sendo muitas vezes preferível reiniciar a sessão. A alucinação, nesse enquadramento, relaciona-se ao esgotamento da capacidade efetiva de raciocínio do modelo diante de excesso de conteúdo. A boa prática, por conseguinte, consiste em decompor o trabalho em tarefas menores, deixando margem para que o modelo erre e se corrija dentro de um ciclo, e em gerenciar ativamente o contexto por compactação ou reinício.

4.4 Persistência de estado entre sessões

O gerenciamento de contexto impõe um problema correlato: como preservar o aprendizado de uma sessão ao reiniciá-la. Uma técnica simples e eficaz é manter um arquivo de rastreamento — um índice de tarefas em formato textual — no qual o agente registra o que foi executado, onde parou e o que resta fazer, marcando o progresso. Assim, ao iniciar nova sessão, o agente retoma a partir do último ponto registrado. Essa externalização do estado para artefatos do projeto é mais robusta do que confiar exclusivamente à memória volátil da janela de contexto.

4.5 Model Context Protocol (MCP)

Modelos de linguagem, isoladamente, não acessam sistemas externos. Conectá-los a uma fonte de dados ou a uma ferramenta exigiria, sem padronização, que o modelo conhecesse a documentação de cada API, lidasse com autenticação e construísse chamadas específicas — tarefa custosa, frágil e que consome contexto. O Model Context Protocol (MCP), proposto como padrão aberto pela Anthropic (ANTHROPIC, 2024), responde a esse problema ao introduzir uma camada intermediária padronizada entre o modelo e os sistemas externos.

Em vez de chamar diretamente a API de um serviço, o agente comunica-se com um servidor MCP que expõe três tipos de recurso: ferramentas (tools), que executam operações

de leitura ou escrita; recursos (resources), que disponibilizam dados; e prompts, que oferecem modelos de instrução. O fluxo típico envolve um processo de descoberta (discovery), no qual o modelo consulta as ferramentas disponíveis e seus parâmetros, seleciona a adequada à tarefa e solicita ao agente que a invoque; o agente executa a chamada ao servidor e devolve o resultado ao modelo, que decide o próximo passo. O protocolo padroniza essa interação, conferindo interoperabilidade e dispensando o tratamento ad hoc de protocolos diversos. Vale notar que mesmo operações locais, como ler ou escrever um arquivo na máquina do desenvolvedor, são modeladas como ferramentas; o MCP é, nesse sentido, um mecanismo geral de extensão das capacidades do modelo.

4.6 Skills: conhecimento estendido e modular

Tanto a inclusão de servidores MCP quanto a manutenção de arquivos de regras extensos consomem tokens, pressionando a janela de contexto. As skills surgem para complementar o MCP e mitigar esse custo, oferecendo conhecimento estendido de forma modular e sob demanda. Uma skill organiza-se em torno de um arquivo descritor conciso — análogo a um sumário — que referencia documentos auxiliares. Apenas o descritor é carregado de início; os documentos detalhados são acessados somente quando a tarefa específica os requer. Desse modo, uma skill pode ser volumosa sem onerar permanentemente o contexto, pois o modelo carrega apenas o fragmento necessário.

Skills podem, ainda, conter código executável e, originadas como recurso de uma ferramenta específica, evoluíram para padrão aberto, com repositórios públicos de distribuição. Exemplos de uso incluem guias de boas práticas de frameworks, rotinas de implementação de interfaces a partir de ferramentas de design, automação de testes de ponta a ponta e skills de entrevista que conduzem o usuário, por meio de perguntas sucessivas, até um plano satisfatório. A escolha entre representar uma capacidade como skill, como servidor MCP ou como subagente nem sempre é unívoca, havendo sobreposição entre os três mecanismos — questão retomada no Capítulo 8.

4.7 Estudo ilustrativo: a evolução do desenvolvimento de interfaces

A evolução do desenvolvimento de interfaces de usuário oferece um caso concreto em que a engenharia de contexto, o MCP e as skills se articulam. Podem-se distinguir três eras. Na era anterior à IA, o fluxo era marcadamente sequencial: um designer produzia as telas em

uma ferramenta de design; seguia-se a transferência de responsabilidade (handoff) ao desenvolvedor, que media espaçamentos e cores, traduzia valores em variáveis de estilo — os chamados tokens de design — e implementava manualmente; o resultado, frequentemente divergente do projeto original, retornava a ciclos de revisão até a conformidade. O processo consumia dias ou semanas e era custoso.

Na primeira era da IA, a integração entre a ferramenta de design e o agente, mediada por um servidor MCP, permitiu extrair do projeto o contexto necessário — variáveis, capturas, estrutura — e gerar automaticamente o código da interface. A burocracia da transferência manual foi eliminada, e o próprio desenvolvedor passou a obter, com poucas instruções, a implementação validada no navegador, em fração do tempo anterior. Essa era pressupõe, contudo, um design tokenizado e estruturado: quanto mais o sistema de design referencia valores reutilizáveis — primitivos, temas, componentes — em vez de valores fixos no código, melhor a IA reproduz e mantém a interface de forma consistente.

Na terceira era, designada de agente como cidadão de primeira classe (first-class), o agente deixa de apenas ler o projeto de design para também escrevê-lo e manipulá-lo, tornando-o uma fonte da verdade mantida pela própria IA. Um humano — desenvolvedor, designer ou mesmo pessoa não técnica — descreve a intenção (um documento de requisitos, um chamado, um prompt), e o agente lê o código, as convenções e as skills, gera as telas no padrão do sistema de design, reutiliza os tokens e implementa o código com testes, validando-o. O custo reduz-se a uma fração do das eras anteriores. O exemplo evidencia dois pontos centrais desta monografia: a expansão de competências propiciada pela IA — um profissional antes restrito ao back-end passa a implementar interfaces com qualidade — e a função do harness, discutida no próximo capítulo, como a base que o desenvolvedor constrói para que outros, inclusive não técnicos, produzam com segurança.

4.8 Validação assistida por ferramentas

A engenharia de contexto também habilita ciclos de validação. Conectando o agente a um navegador por meio de MCP, é possível capturar imagens da interface em execução e compará-las, pixel a pixel, com a especificação visual de origem, gerando relatórios de divergência (drift). Analogamente, ferramentas de automação de navegador permitem que o agente percorra a aplicação, preencha formulários, submeta entradas inválidas e proponha testes de ponta a ponta. Tais recursos transformam a validação — historicamente manual e

cara em escala — em etapa automatizável do fluxo, antecipando a discussão sobre laços de realimentação do próximo capítulo.

5 Engenharia de Ambiente Operacional (Harness)

5.1 As três camadas: prompt, contexto e harness

Os Capítulos 3 e 4 trataram, respectivamente, do que se pede ao modelo (prompt) e do que o modelo vê (contexto). Há, porém, uma terceira camada, igualmente decisiva: o ambiente em que o agente opera. A essa camada dá-se o nome de harness — termo sem tradução literal consagrada, aproximável da ideia de arreamento ou de infraestrutura operacional. A engenharia de ambiente operacional (harness engineering) é a disciplina que projeta esse ambiente.

A analogia mais difundida é a do cavalo e da montaria: um modelo de linguagem isolado é como um cavalo sem sela e arreios — potente, porém impossível de conduzir com segurança. Outra imagem recorrente é a do trabalho em altura: o operador competente não escala a fachada de um edifício a mão livre, mas se vale de cordas, cintos e equipamentos que lhe permitem executar a tarefa com segurança. O harness é esse conjunto de recursos que envolve o modelo e o capacita a agir. Não se trata da infraestrutura computacional de hardware, mas do aparato lógico — instruções, ferramentas, regras e laços de verificação — que disciplina a atuação do agente.

5.2 Harness embutido e harness do projeto

Convém distinguir dois níveis de harness. O primeiro é o harness embutido, fornecido pela própria ferramenta agêntica de codificação: o prompt de sistema que acompanha cada interação, os recursos de criação, bifurcação e compactação de sessão, e a instrumentação que reporta o consumo de contexto. Esse harness é condição de funcionamento, pois o modelo, por si só, não sabe operar.

O segundo é o harness do projeto, construído pelo próprio time para adequar o agente às particularidades de seu sistema. Compõem-no os arquivos de regras e diretrizes — tipicamente um arquivo de guidelines carregado em toda sessão —, as skills específicas, os servidores MCP pertinentes e, sobretudo, os laços de realimentação que verificam o trabalho da IA. É o harness do projeto que transforma uma ferramenta genérica de geração de código

em um colaborador alinhado às convenções, às restrições e à arquitetura de um sistema concreto, seja ele novo ou legado.

5.3 Arquivos de diretrizes e regras

O arquivo de diretrizes do projeto é o documento mais importante do harness, pois é carregado em todas as sessões. Nele se registram as decisões gerais e estáveis: a pilha tecnológica, as convenções de arquitetura, a exigência de testes, a separação de camadas. Justamente por ser sempre carregado, deve ser conciso — da ordem de poucas centenas de linhas. Inflá-lo com detalhes específicos desperdiça contexto em toda interação. Para instruções de aplicação restrita — por exemplo, o padrão a seguir na escrita de testes — empregam-se arquivos de regras carregados sob demanda, ativados apenas no momento pertinente. Essa segmentação é, ela própria, uma prática de engenharia de contexto aplicada à governança do projeto.

5.4 Laços de realimentação (feedback loops)

O elemento que distingue um harness maduro é a presença de laços de realimentação que verificam, automaticamente, o resultado produzido pela IA. Após a implementação de uma funcionalidade, o ambiente pode disparar a execução de testes, a análise estática, a verificação de formatação, a conferência de que os arquivos criados respeitam os padrões do projeto e a checagem de que as fronteiras entre camadas não foram violadas. Também na preparação do ambiente, antes do início de uma tarefa, scripts de inicialização podem ser invocados.

Um padrão ilustrativo é o de uma "porta de testes" (test gate): um script que executa as baterias de teste e, em caso de falha, concede ao agente um número limitado de tentativas de autocorreção antes de sinalizar erro. Esse mecanismo de realimentação para a frente (feed-forward) faz com que o agente execute, observe o resultado e se realimente até atingir o critério de sucesso. Pode-se ainda incorporar verificações mais sofisticadas, como a comparação de capturas de tela com a especificação visual ou a análise de logs. O princípio é geral: qualquer verificação que assegure qualidade pode ser embutida como gatilho posterior à implementação.

5.5 Segurança no ambiente operacional

O harness é também o lugar natural para controles de segurança operacional. Cabe ao ambiente impedir que o agente acesse credenciais sensíveis — chaves de acesso, arquivos de configuração de infraestrutura, segredos de repositórios — e restringir operações potencialmente destrutivas a confirmações explícitas. A configuração cuidadosa dessas permissões integra a fundação que permite executar tarefas com segurança, sobretudo quando múltiplos agentes operam de forma autônoma. A discussão aprofundada de segurança é retomada no Capítulo 9.

5.6 Síntese: o harness como camada de governança

Em síntese, a engenharia de ambiente operacional é a camada que converte a potência bruta do modelo em produção confiável. Articulada à engenharia de prompt e à engenharia de contexto, ela fecha o tripé conceitual proposto nesta monografia. Enquanto o prompt e o contexto determinam, respectivamente, a instrução e a informação, o harness determina as condições, as verificações e os limites da ação — sendo, por isso, a disciplina mais diretamente associada à governança da automação.

6 Desenvolvimento Orientado a Especificações

6.1 Motivação: os limites do planejamento livre

O desenvolvimento orientado a especificações (Spec-Driven Development — SDD) inscreve-se na tradição de siglas que associam uma prática de engenharia a um eixo orientador, como o projeto orientado ao domínio (DDD) e o desenvolvimento guiado por testes (TDD). Sua motivação nasce dos limites do planejamento livre na interação com a IA. Um padrão difundido é o RPI — pesquisar, planejar, implementar (research, plan, implement) —, em que se antecede a implementação por uma fase de exploração e por outra de planejamento. Contudo, quando o planejamento é conduzido de forma não estruturada, surgem problemas recorrentes.

- Ambiguidade: instruções vagas (por exemplo, "adicionar um sistema de cache") não explicitam intenção, motivação nem problema a resolver, levando o agente a implementar algo desalinhado.

- Ausência de critério de validação: sem critérios definidos, não há como auditar se o que foi feito corresponde ao que se pretendia.
- Desvio de escopo (scope drift): a implementação diverge da especificação inicial.
- Acoplamento entre planejamento e execução: o mesmo agente que planeja, em alto nível, executa em baixo nível, sem a separação de responsabilidades que beneficiaria ambas as fases.
- Perda de contexto entre rodadas: ao reiniciar a sessão, o agente pode desfazer o que havia sido feito, por não preservar o estado das decisões.

O SDD responde a esses problemas ao tornar a especificação o eixo do processo. Ela deixa de ser um texto descartável e passa a ser a fonte da verdade, contra a qual o resultado é continuamente confrontado.

6.2 A esteira de refinamento

O SDD pode ser compreendido como uma esteira de refinamento progressivo. Parte-se de uma intenção; segue-se um planejamento; validam-se os critérios; confrontam-se as decisões com especificações e planos anteriores; decompõe-se o trabalho em tarefas granulares; executa-se; e, ao final, reavalia-se se o produzido está alinhado à especificação de origem. O erro mais comum no desenvolvimento assistido por IA é acelerar a implementação e subvalorizar o planejamento. Sob o novo paradigma, a implementação é um detalhe a jusante: dado um planejamento bem feito, refinado, validado e ancorado em contratos, a IA produz boa implementação em poucos minutos. O papel do profissional é, novamente, o de conceber para que a ferramenta construa, revisando e auditando em seguida.

6.3 Ferramentas de SDD e o conceito de constituição

O SDD não é nativo de nenhuma ferramenta de IA; é um arcabouço que pode ser implementado de diversas formas, materializado em uma sequência de documentos que progridem até o plano de implementação. Existem implementações públicas variadas; uma das mais conhecidas é mantida pela equipe responsável por uma plataforma de hospedagem de código e é compatível com as principais ferramentas agênticas. A instalação habilita um conjunto de comandos que conduzem o ciclo: estabelecer a constituição do projeto, especificar, planejar, pesquisar, derivar tarefas, analisar consistência e implementar.

A constituição é o documento inaugural e mais estável: nela se fixam princípios inegociáveis — qualidade de código, integridade de contratos, política de testes, experiência do usuário, desempenho. Toda nova especificação é confrontada com a constituição; havendo contradição, há três caminhos: alterar a constituição e então prosseguir, prosseguir com ressalva explícita (caso indesejável) ou abortar e reavaliar. Diferentemente da constituição, que muda apenas por revisão deliberada, os demais documentos evoluem conforme a necessidade do projeto. O processo é agnóstico quanto a documentos de apoio: integra-se naturalmente a registros de decisão de arquitetura, documentos de requisitos, histórias de usuário e épicas, conforme a cultura da equipe.

6.4 A malha de documentos gerada

Uma única especificação engendra uma malha de artefatos: a especificação propriamente dita, com cenários e critérios de aceitação; o plano técnico, com dependências, restrições e contexto; a pesquisa de alternativas; o modelo de dados; os contratos — por exemplo, o esquema de uma API ou de variáveis de ambiente, posteriormente validados contra a implementação; e o conjunto de tarefas granulares, organizadas em fases. A presença de critérios de aceitação explícitos e testáveis é o que viabiliza, ao final, a reverificação do que foi construído. Cumpre sublinhar um limite: o SDD não promete que a IA construa sozinha um software de qualidade. Ele é apenas a metodologia de planejamento e execução; se o processo de concepção do software é deficiente, a especificação resultante também o será, e a implementação herdará o defeito. O SDD é, portanto, uma parte do processo, que se articula às demais disciplinas aqui discutidas.

6.5 Rastreamento de progresso e gestão de contexto

A decomposição em tarefas granulares, com marcação explícita de progresso, conecta o SDD à gestão de contexto discutida no Capítulo 4. Como o estado das tarefas concluídas é registrado em artefatos, não é necessário executar todas as tarefas em uma única sessão: pode-se compactar o histórico ou reiniciar a sessão e retomar a partir da última tarefa concluída. Essa disciplina evita que a janela de contexto se esgote e que o agente, sobrecarregado, passe a produzir resultados de baixa qualidade.

7 Paralelização do Trabalho e Automação do Ciclo de Entrega

7.1 Do desenvolvimento sequencial ao paralelo

No modelo tradicional, uma funcionalidade percorre um fluxo sequencial conduzido por um único desenvolvedor: conceber, escrever, testar, depurar, abrir a requisição de incorporação, revisar e integrar. Esse ciclo, para uma única funcionalidade, consome de dias a semanas. O novo paradigma habilita a paralelização: o profissional, no papel de orquestrador, especifica diversas funcionalidades e atribui cada uma a um agente distinto, que as desenvolve simultaneamente. O ciclo de revisão, decisão e integração permanece, agora sob a forma de coautoria com humano no laço (human-in-the-loop), mas o resultado é a entrega de múltiplas funcionalidades no intervalo antes ocupado por uma só.

A paralelização não dispensa o discernimento de engenharia. Tarefas devem ser classificadas quanto à criticidade: defeitos simples ou de baixo risco podem ser tratados com elevado grau de automação — por exemplo, um erro capturado por uma ferramenta de monitoramento dispara um agente que abre uma correção e a submete —, enquanto problemas complexos exigem maior supervisão humana. O critério de risco é o que governa o grau admissível de autonomia.

7.2 Árvores de trabalho do Git (git worktrees)

A execução de múltiplos agentes sobre a mesma base de código esbarra em uma limitação fundamental do sistema de controle de versão: por padrão, trabalha-se em uma única ramificação por vez em um dado diretório. A solução adequada é o recurso de árvores de trabalho (worktrees) do Git, que permite manter clones do projeto associados a ramificações distintas, cada qual em seu próprio diretório. Trata-se de funcionalidade consolidada do Git, preferível ao uso de armazenamento temporário (stash) para alternar entre frentes de trabalho. Com worktrees, cada agente opera em um repositório isolado, eliminando a interferência mútua na manipulação de arquivos.

A coordenação de múltiplos ambientes reintroduz preocupações de engenharia que recaem sobre o harness. Aplicações executadas em paralelo disputam recursos como portas de rede e instâncias de banco de dados; é necessário, portanto, prover scripts de inicialização que dinamizem portas e isolem bancos, permitindo que os agentes compartilhem recursos sem fricção. Resta, ainda, o desafio inevitável da integração: conflitos de mesclagem ocorrerão e

devem ser resolvidos com as técnicas usuais de engenharia, eventualmente apoiadas por sinalizadores de funcionalidade (feature flags) para desacoplar a entrega da ativação.

7.3 Planejamento de dependências entre tarefas

A paralelização eficaz pressupõe um mapeamento explícito de dependências. Modelar-se o conjunto de tarefas como um grafo, distinguindo tarefas bloqueantes — que devem ser concluídas antes de outras — das que podem progredir em paralelo. Esse roteiro orienta a distribuição do trabalho entre agentes, assegurando que apenas tarefas efetivamente independentes sejam executadas simultaneamente. O planejamento de dependências é, portanto, condição da paralelização segura.

7.4 Agentes de revisão e automação da integração

A automação estende-se à revisão de código. É possível configurar, no fluxo de integração contínua, agentes que revisam automaticamente cada requisição de incorporação. Tais agentes avaliam qualidade de código, identificam oportunidades de refatoração e, por estarem frequentemente conectados a bases de vulnerabilidades conhecidas, examinam dependências e trechos potencialmente inseguros — verificações por vezes difíceis de realizar na máquina local. Distinguem-se dois arranjos: um agente de revisão genérico, que não conhece o harness do projeto, e um agente configurado pelo time, que carrega as regras, skills e convenções específicas e, por isso, mantém maior aderência ao contexto, podendo inclusive propor ou aplicar correções.

No limite, compõe-se um fluxo amplamente automatizado: um chamado registrado em uma ferramenta de gestão converte-se em requisição de incorporação; um agente implementa; outro revisa e confronta o resultado com a constituição do projeto; correções são aplicadas; e a tarefa é marcada como concluída na ferramenta de origem. O humano permanece no laço como autoridade revisora, mas a cadência e o volume de trabalho ampliam-se substancialmente. Esse arranjo materializa, no plano operacional, a figura do orquestrador discutida no Capítulo 2.

8 Arquiteturas de Agentes de Inteligência Artificial

8.1 O que é, e o que não é, um agente

A palavra "agente" tornou-se ubíqua e, por isso mesmo, imprecisa. Convém uma definição rigorosa. Um agente de IA é, antes de tudo, software cujo núcleo de decisão é consumido por um modelo de linguagem. Dessa característica decorrem propriedades essenciais: o comportamento é não determinístico, pois as decisões emanam do modelo; o agente possui um domínio claro, compreendendo seu papel e seu ambiente; dispõe de ferramentas específicas para resolver tarefas, que invoca segundo seu próprio juízo; exibe capacidade de adaptação, lidando com dados não estruturados; e opera com mecanismos de memória, de curto ou de longo prazo. A avaliação sistemática (evaluation) de seu comportamento integra o desenvolvimento desde a concepção.

É fundamental distinguir agente de automação. Uma automação — um fluxo configurado em uma plataforma de orquestração visual, por exemplo — é um workflow determinístico que pode, inclusive, invocar um modelo de linguagem em uma de suas etapas, mas cuja lógica de controle é fixa. Não há, nela, decisão delegada à IA no sentido agêntico. A distinção não é depreciativa: automações resolvem muitos problemas com eficácia. O ponto é classificatório — nem todo fluxo que chama um modelo é um agente.

Decorre daí um princípio de projeto: não se deve introduzir IA em toda parte. Processos inerentemente determinísticos — a confirmação de um pagamento, por exemplo — não se beneficiam de não determinismo. A pertinência do uso de agentes deve ser avaliada caso a caso, evitando-se o impulso, comum no mercado, de agentificar indiscriminadamente.

8.2 Arquitetura de agente único

A arquitetura mais simples é a de agente único: um laço que mantém um contexto e dispõe de um conjunto de ferramentas que pode ou não utilizar. O usuário interage com o agente, que combina prompt de sistema, prompt do usuário e ferramentas — incluindo busca na web, acesso a banco de dados e servidores MCP — para produzir um resultado. Sua principal virtude é a simplicidade de orquestração: como há um único estado e um único fluxo de raciocínio, a gestão da janela de contexto é direta. Seus riscos são igualmente claros: o excesso de ferramentas tende a confundir a escolha do modelo, e o acúmulo de dados em uma única janela amplia a probabilidade de alucinação. Como regra prática, recomenda-se iniciar

todo projeto com um agente único e migrar para arranjos multiagente apenas quando a necessidade se demonstrar.

8.3 Arquitetura multiagente

Na arquitetura multiagente, múltiplas unidades autônomas cooperam. O arranjo típico comporta um agente orquestrador, que planeja, delega e sintetiza, e diversos subagentes especializados, cada qual responsável por um domínio. O usuário dirige-se ao orquestrador, que decide quais subagentes acionar e em que ordem, recebendo de cada um, ao final, um resumo ou o resultado integral. Por analogia, se o agente único evoca o monólito, o arranjo multiagente evoca os microsserviços — com a ressalva de que se trata de analogia didática, e não de equivalência técnica.

Um aspecto crítico distingue subagentes de meras chamadas: cada subagente possui janela de contexto própria e isolada. Essa separação é virtuosa, pois evita a saturação de uma janela única, mas impõe um desafio de coordenação: é preciso decidir quais dados enviar a cada subagente e como sintetizar suas devoluções. Para conter o crescimento do contexto do orquestrador, os subagentes tipicamente retornam resumos, e não a totalidade de seu processamento. Subagentes podem, ainda, acionar outros subagentes, formando uma malha cuja complexidade exige controle cuidadoso, sob pena de laços infinitos — daí a recomendação de mecanismos de tempo-limite (timeout).

8.4 Trade-offs entre as arquiteturas

A escolha arquitetural é, essencialmente, um balanço de trade-offs, sintetizado no Quadro 2. O isolamento de contexto, ganho com a especialização dos subagentes, custa latência e dinheiro. A especialização melhora a qualidade local, mas duplica tokens, pois informação processada por um agente é frequentemente reprocessada por outro. O paralelismo acelera a execução, mas a coordenação mal planejada pode encarecê-la sobremaneira. A observabilidade, simples no agente único, torna-se difícil quando muitas unidades interagem. Há relatos, no mercado, de sistemas multiagente que precisaram ser desativados por planejamento inadequado. A diretriz prática reitera-se: começar com um agente e adotar a multiplicidade apenas quando justificada.

Quadro 2 — Trade-offs entre agente único e arquitetura multiagente.

Dimensão	Ganho	Custo / risco
----------	-------	---------------

Isolamento de contexto	Evita saturação da janela	Maior latência e custo
Especialização	Maior qualidade por domínio	Duplicação de tokens
Paralelismo	Execução simultânea	Coordenação difícil; pode encarecer
Observabilidade	Simples no agente único	Difícil em malhas multiagente
Robustez	Modularidade	Risco de laços infinitos (usar timeout)

Fonte: elaborado pelo autor.

8.5 Economia de tokens em aplicações agênticas

Cabe distinguir o uso de IA no fluxo de desenvolvimento do uso de IA embarcado em aplicações de produção. No primeiro, a tendência de queda de custos e de ampliação das janelas de contexto torna a preocupação com tokens cada vez menos central, desde que não haja desperdício evidente. No segundo, a economia é parte do projeto: não se emprega, em produção, necessariamente o modelo mais capaz e caro. Selecionam-se modelos adequados a cada subtarefa — um modelo eficiente em sumarização para etapas de resumo, por exemplo — e submete-se a escolha a processos de avaliação que medem tanto custo quanto qualidade do resultado. Camadas de roteamento de modelos (LLM gateways) auxiliam ao selecionar, dinamicamente, o modelo mais apropriado a cada chamada.

8.6 Frameworks de desenvolvimento de agentes

Não se constrói um agente do zero; parte-se de um framework. Dois exemplos ilustram abordagens distintas. O primeiro, de orientação "código primeiro" (code-first) e mantido por um grande provedor de nuvem, amadureceu internamente antes de ser liberado como código aberto e sustenta produtos em produção. Oferece kits de desenvolvimento em várias linguagens e privilegia a facilidade de composição: criam-se agentes, acoplam-se ferramentas e o sistema opera. Seus componentes essenciais são: o agente base, em variantes como agente de modelo, sequencial, paralelo e de laço; as sessões e o estado, com objetos mutáveis e imutáveis; as ferramentas, externas ou pré-existentes; os ganchos (hooks), acionados por ações; e os executores e eventos, que orquestram a execução.

O segundo framework adota a abordagem de grafos: o fluxo entre agentes é definido como um grafo de dependências, no qual nós representam agentes e arestas, as transições.

Essa formalização — próxima da noção de autômatos — confere controle fino sobre o encaminhamento (handoff) da informação entre unidades, ao custo de uma curva de aprendizado mais acentuada. Sua pilha principal é a linguagem Python. Há, ainda, outros frameworks relevantes, com focos distintos, inclusive opções voltadas ao ecossistema TypeScript. A escolha depende do equilíbrio desejado entre simplicidade de composição e controle explícito do fluxo.

8.7 O padrão agente-como-ferramenta

Um padrão recorrente em arquiteturas multiagente é o de agente-como-ferramenta (agent-as-tool): um agente é exposto a outro como se fosse uma ferramenta invocável. Assim, o orquestrador não distingue, em sua lógica de chamada, entre uma ferramenta convencional e um subagente especializado; ambos são acionados conforme a necessidade. Esse padrão simplifica a composição e reforça a especialização, pois cada subagente encapsula competências — e possivelmente suas próprias skills e servidores MCP — que o orquestrador não precisa conhecer em detalhe.

8.8 Geração aumentada por recuperação (RAG)

Aplicações agênticas frequentemente precisam operar sobre grandes volumes de documentos — um agente jurídico, por exemplo, pode lidar com processos de centenas a milhares de páginas. Carregar tais documentos integralmente no contexto é inviável e contraproducente. A geração aumentada por recuperação (Retrieval-Augmented Generation — RAG), formalizada por Lewis et al. (2020), resolve o problema ao recuperar, sob demanda, apenas os fragmentos relevantes a cada consulta, agregando-os ao contexto do modelo. O RAG é, nesse sentido, engenharia de contexto aplicada a bases de conhecimento extensas.

O pipeline de RAG compreende duas fases. Na ingestão, os documentos de origem são extraídos, segmentados em fragmentos (chunks) e convertidos em vetores por um modelo de embedding, sendo armazenados em um banco vetorial ou em um índice com suporte a busca vetorial. A segmentação não é trivial: define-se uma sobreposição (overlap) entre fragmentos contíguos para preservar o contexto e evitar que trechos sejam recuperados fora de seu sentido — por exemplo, uma afirmação que, isolada de sua oração condicional, inverteria o significado pretendido. Na recuperação, a consulta do usuário é convertida em vetor e comparada, por similaridade — tipicamente a similaridade de cosseno —, aos vetores

armazenados, retornando os fragmentos semanticamente mais próximos. A busca semântica supera a busca léxica por tolerar variações de forma e capturar proximidade de sentido. Recupera-se, assim, somente o necessário, em consonância com o princípio de manter o contexto enxuto.

8.9 Ilustração: aplicação multiagente com RAG

A título de ilustração, considere-se uma aplicação multiagente hipotética de apoio jurídico. Uma interface de conversação comunica-se com um serviço escrito em Python, no qual reside o framework de agentes. Um agente orquestrador recebe a solicitação e delega a subagentes especializados: um agente de pesquisa, que consulta a base interna de processos; um agente de jurisprudência, que pesquisa decisões na base interna e em fontes externas, mediadas por uma ferramenta de busca; e um subagente de análise, que consolida processos similares e jurisprudências aplicáveis para sugerir próximos passos. Os processos, documentos volumosos e pouco estruturados, são previamente ingeridos por um pipeline de RAG e armazenados de forma vetorial, permitindo recuperação semântica precisa. Esse exemplo evidencia como as disciplinas discutidas — engenharia de contexto, RAG, arquitetura multiagente e seleção de modelos — se integram em um sistema coeso, e como os novos softwares deslocam a interação do usuário de formulários rígidos para o diálogo orientado por intenção.

8.10 Memória e a colisão entre mecanismos

A memória dos agentes admite implementações diversas, com trade-offs próprios. Pode-se persistir um resumo ao final de cada interação, por meio de ganchos, alimentando um registro de mudanças; pode-se confiar a memória compartilhada da ferramenta; ou pode-se evitar a memória persistente em favor de laços de realimentação que revalidem o trabalho a cada execução. Cada opção pressiona a janela de contexto de modo distinto: a memória antecipa correções, mas consome contexto cumulativamente; a validação posterior preserva contexto, mas pode consumir mais tokens em reexecução. Não há escolha universalmente superior, apenas trade-offs a ponderar na engenharia de contexto.

Por fim, registre-se a colisão entre três mecanismos de extensão — skills, subagentes e servidores MCP —, que por vezes podem cumprir funções semelhantes. A distinção orientadora é a seguinte: a skill provê conhecimento estendido, modular e carregado sob

demanda, no mesmo contexto do agente que a aciona; o subagente oferece janela de contexto distinta, favorecendo paralelismo e especialização; e o servidor MCP padroniza o acesso a sistemas e dados externos. Um subagente pode, internamente, valer-se de skills e de servidores MCP sem que o orquestrador o saiba. A decisão entre os três depende de se o objetivo é estender conhecimento, isolar contexto ou integrar sistemas.

9 Segurança, Qualidade e Governança

9.1 Novos vetores de ameaça

A introdução de modelos de linguagem no ciclo de software cria classes de vulnerabilidade que não constituem meros defeitos pontuais, mas frentes inteiras a serem dominadas. Destacam-se a injeção de prompt (prompt injection), na qual instruções maliciosas embutidas em dados de entrada subvertem o comportamento do modelo; o jailbreaking, que busca contornar as restrições de segurança; e os ataques à cadeia de suprimentos (supply chain), nos quais dependências comprometidas inserem comportamento indesejado. Há casos documentados de injeção de prompt por meio de texto oculto em documentos — instruções inseridas em fonte de cor imperceptível, destinadas a manipular sistemas que processam o conteúdo — e de comprometimento de fluxos de integração por dependências adulteradas. Tais ameaças exigem que o desenvolvedor, responsável por construir os sistemas que outros utilizarão, internalize a segurança como requisito de projeto.

9.2 Guardrails

A resposta a esses vetores articula-se em torno de guardrails — proteções interpostas entre a entrada, o modelo e a saída. Guardrails podem operar em diferentes níveis: no nível do prompt, restringindo comportamentos; no nível determinístico do código, validando entradas e saídas; e por meio de ferramentas dedicadas. Para a proteção de dados sensíveis — senhas, números de cartão de crédito —, uma estratégia eficaz é o emprego de um modelo auxiliar de baixo custo que sanitize a entrada, removendo informação sigilosa antes que ela alcance o modelo principal, eventualmente executado em infraestrutura própria para evitar a exposição do dado a terceiros. Validações determinísticas — expressões regulares, pipelines de filtragem oferecidos por frameworks — complementam a defesa. O padrão de empregar um segundo

modelo como avaliador (LLM-as-a-judge) também se presta a verificar a conformidade de entradas e saídas.

9.3 Qualidade, avaliação e custo

A qualidade, na era da IA, abrange tanto a qualidade do código quanto a do resultado produzido pelo modelo. Para aferi-la, adotam-se processos explícitos de avaliação (evaluation), que mensuram a adequação das respostas, e métricas de latência e custo, que determinam a viabilidade econômica da inclusão de IA em um processo. A latência, em particular, impõe um novo regime de pensamento: chamadas a modelos podem demorar dezenas de segundos a minutos, ordens de grandeza acima das latências de milissegundos a que a arquitetura de software convencional está habituada. Decisões de projeto — síncrono versus assíncrono, escolha de modelo, uso de cache — passam a ponderar esse fator.

9.4 Observabilidade e governança da automação

Quanto maior a autonomia e a multiplicidade de agentes, maior a exigência de observabilidade. Em arranjos multiagente, falhas de coordenação — laços infinitos, duplicação de trabalho — podem escalar custos e comprometer a confiabilidade. Instrumentar a execução, registrar trajetórias de decisão e versionar prompts e skills são práticas que sustentam a governança. Plataformas de observabilidade de agentes e de gestão de prompts auxiliam nesse controle, permitindo versionamento, testes e análise de consumo. A governança da automação é, em última análise, a contraparte responsável do ganho de produtividade: sem ela, a escala que beneficia também amplifica os danos.

10 Implicações para a Formação e o Mercado de Trabalho

10.1 Fundamentos e o desafio do profissional iniciante

A automação da escrita de código cria um paradoxo formativo. Os fundamentos — estruturas de dados, algoritmos, arquitetura, princípios de projeto — tornam-se mais necessários, pois habilitam a especificar, revisar e validar; ao mesmo tempo, a ferramenta oferece todos os incentivos para que o iniciante não os aprenda, entregando resultados imediatos que dispensam, na superfície, a compreensão profunda. O profissional experiente

surfa a nova onda com vantagem, porque dispõe da fundamentação que lhe permite julgar o que a IA produz; o iniciante enfrenta o risco de construir sobre alicerces frágeis.

A recomendação que se depreende é a do balanceamento: estudar, simultaneamente, os fundamentos e as práticas de IA, sem o radicalismo de negligenciar qualquer dos polos. Negligenciar os fundamentos compromete a capacidade de auditar; ignorar a IA significa ausentar-se de um movimento já consolidado no mercado. A proporção entre ambos varia conforme a trajetória de cada profissional, mas a paralelização dos dois estudos é condição para a relevância futura.

10.2 Reciclagem contínua e ruído informacional

A velocidade de mudança do campo impõe reciclagem contínua. Conceitos centrais hoje — janelas de contexto de grande capacidade, skills, harness — eram inexistentes ou incipientes há pouco tempo. Parte do conhecimento técnico específico terá de ser descartada e renovada, enquanto os fundamentos permanecem. Soma-se a isso o ruído informacional: a profusão de relatos exagerados em redes sociais alimenta ansiedade e comparação improdutivo. A postura recomendável combina constância de estudo com discernimento crítico quanto às fontes, distinguindo demonstrações substantivas de autopromoção.

10.3 Adoção proporcional e contexto econômico

A adoção de IA não precisa ser total para ser benéfica. Organizações com recursos amplos podem incorporar IA em todo o fluxo de desenvolvimento — planejamento, implementação, revisão automatizada; outras, com restrições orçamentárias, podem adotá-la de modo mais pontual, em depuração, exploração e refatoração, nas quais o custo é mais contido. Há relatos de empresas que, avaliando seu contexto, optaram por usos mais restritos por considerarem o custo da adoção massiva injustificado. O ponto de equilíbrio é específico de cada organização; o que não se sustenta é a ausência completa de uso, dado o diferencial competitivo que a ferramenta oferece. Aplicações de tecnologias legadas, inclusive, beneficiam-se: a documentação relevante pode ser incorporada ao projeto, em formato apropriado, ou encapsulada em skills, habilitando o modelo a operar mesmo sobre pilhas antigas.

11 Considerações Finais

Esta monografia propôs-se a sistematizar, em chave de engenharia, o campo em formação do desenvolvimento de software assistido por inteligência artificial. O percurso partiu da redefinição do papel do desenvolvedor — de codificador a coautor e orquestrador — e do deslocamento da complexidade do código para a especificação, e organizou o aparato conceitual em três disciplinas complementares: a engenharia de prompt, que cuida da instrução; a engenharia de contexto, que cuida da informação disponível ao modelo; e a engenharia de ambiente operacional (harness), que cuida das condições, verificações e limites da ação. Sobre esse tripé articularam-se o desenvolvimento orientado a especificações, a paralelização do trabalho e as arquiteturas de agentes, incluindo a geração aumentada por recuperação.

Dois resultados merecem destaque. O primeiro é que a IA não dispensa, mas intensifica, a exigência de fundamentos sólidos de engenharia de software: é o domínio desses fundamentos que habilita o profissional a especificar com precisão, a auditar com rigor e a responder pela qualidade do que a ferramenta produz. O segundo é que cada decisão nesse novo regime — escolha de técnica de prompting, gestão de contexto, arquitetura de agentes, estratégia de memória — constitui um balanço de trade-offs, e não a aplicação de uma solução universal. A própria noção de "bala de prata", recusada pela engenharia de software clássica, permanece ausente.

Reconhecem-se as limitações do estudo. Por sua natureza teórico-conceitual e pela velocidade de evolução do campo, parte das ferramentas e dos números aqui mencionados estará desatualizada em curto prazo; os conceitos estruturantes, contudo, tendem a maior permanência. Não se realizaram experimentos controlados de medição de produtividade ou de qualidade, o que delimita o alcance das afirmações a uma sistematização interpretativa.

Como trabalhos futuros, sugerem-se: estudos empíricos que quantifiquem o impacto das práticas descritas sobre produtividade, qualidade e custo em contextos reais; a investigação de métricas e benchmarks de avaliação de agentes; o aprofundamento das questões de segurança específicas de sistemas agênticos; e a análise dos efeitos de longo prazo sobre a formação de profissionais iniciantes. A consolidação de um corpo conceitual maduro, ao qual esta monografia pretende contribuir, é condição para que a engenharia de software assistida por IA seja conduzida com o rigor que a disciplina sempre exigiu.

Referências

- ANTHROPIC. Introducing the Model Context Protocol. 2024. Disponível em: <https://www.anthropic.com/news/model-context-protocol>. Acesso em: 2026.
- BECK, K. Test-Driven Development: By Example. Boston: Addison-Wesley, 2002.
- BROWN, T. B. et al. Language Models are Few-Shot Learners. In: Advances in Neural Information Processing Systems (NeurIPS), v. 33, 2020.
- EVANS, E. Domain-Driven Design: Tackling Complexity in the Heart of Software. Boston: Addison-Wesley, 2003.
- GAMMA, E.; HELM, R.; JOHNSON, R.; VLISSIDES, J. Design Patterns: Elements of Reusable Object-Oriented Software. Reading: Addison-Wesley, 1994.
- HUMANLAYER. 12-Factor Agents: Principles for building reliable LLM applications. 2024. Disponível em: <https://github.com/humanlayer/12-factor-agents>. Acesso em: 2026.
- LEWIS, J.; FOWLER, M. Microservices: a definition of this new architectural term. 2014. Disponível em: <https://martinfowler.com/articles/microservices.html>. Acesso em: 2026.
- LEWIS, P. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In: Advances in Neural Information Processing Systems (NeurIPS), v. 33, 2020.
- LIU, N. F. et al. Lost in the Middle: How Language Models Use Long Contexts. Transactions of the Association for Computational Linguistics, 2023.
- MARTIN, R. C. Clean Code: A Handbook of Agile Software Craftsmanship. Upper Saddle River: Prentice Hall, 2008.
- MARTIN, R. C. Clean Architecture: A Craftsman's Guide to Software Structure and Design. Upper Saddle River: Prentice Hall, 2017.
- SCHULHOFF, S. et al. The Prompt Report: A Systematic Survey of Prompting Techniques. arXiv:2406.06608, 2024.
- VASWANI, A. et al. Attention Is All You Need. In: Advances in Neural Information Processing Systems (NeurIPS), v. 30, 2017.
- WEI, J. et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In: Advances in Neural Information Processing Systems (NeurIPS), v. 35, 2022.
- WIGGINS, A. The Twelve-Factor App. 2011. Disponível em: <https://12factor.net>. Acesso em: 2026.
- YAO, S. et al. ReAct: Synergizing Reasoning and Acting in Language Models. In: International Conference on Learning Representations (ICLR), 2023.